

LABORATORY WORK #3

Data Manipulation Language (DML)

PostgreSQL Database: airport_lab1

Execution evidence: SQL statements and before/after screenshots

Student Kassymkozha Ergali

Institution Kazakhstan British Technical University

Course Databases

Task Overview

The report is organized in the same order as the assignment. Each section identifies the operation, shows the relevant SQL execution evidence, and includes before/after verification where it was captured.

Task	Operation
1	Generate and insert 200 random rows in the airport database.
2	Add a new airline named "KazAir" based in "Kazakhstan" to the airline table.
3	Update the airline country for "KazAir" to "Turkey".
4	Add three airlines at once: AirEasy (France), FlyHigh (Brazil), and FlyFly (Poland).
5	Delete all flights whose scheduled arrival is in the year 2024.
6	Increase the price of all tickets in the booking table by 15%.
7	Delete all tickets where the price is less than 10000.
8	Set airline_code to "UNK" where airline_code is NULL.
9	Delete baggage check records created before June 1, 2023 whose result is "Not checked".
10	Delete airport entries where state is NULL and city is either Mlawe or Kepuh.
11	Insert a new unchecked baggage inspection and return the generated baggage_check_id and created_at.
12	Convert all country names in the airline table to uppercase.
13	Update airline_id = 5 to Global Airways, United Kingdom, and set updated_at to the current time.
14	Set state to "Capital District" for airports in Astana, London, and Tokyo.
15	Mark March 2024 baggage checks with status "Not checked" as "Checked".

Task 1

Generate and insert 200 random rows in the airport database.

SQL statement

```
1  INSERT INTO airport_lab1.airport (
2      airport_name,
3      country,
4      state,
5      city,
6      created_at,
7      updated_at
8  )
9  SELECT
10     airport_name 'Test Airport ' || gs,
11     country (ARRAY['Kazakhstan', 'France', 'Turkey', 'USA', 'Japan'])
12         [1 + floor(random() * 5)::int],
13     state (ARRAY['Almaty Region', 'California', 'London', 'Tokyo'])
14         [1 + floor(random() * 4)::int],
15     city (ARRAY['Almaty', 'Paris', 'Istanbul', 'London', 'Tokyo'])
16         [1 + floor(random() * 5)::int],
17     created_at NOW(),
18     updated_at NOW()
19 FROM generate_series(1, 200) AS gs;
```

INSERT ... SELECT with generate_series(1, 200) and random values.

Verification query

```
1  ✓ SELECT * 343 ms
2  FROM airport_lab1.airport
3  ORDER BY airport_id DESC
4  LIMIT 10;
```

The latest airport rows are selected in descending airport_id order.

Result / after

	airport_id	airport_name	country	state	city	created_at
1	200	Test Airport 200	USA	Almaty Region	Tokyo	2026-09-29 19:07
2	199	Test Airport 199	Japan	California	London	2026-09-29 19:07
3	198	Test Airport 198	Japan	Almaty Region	Paris	2026-09-29 19:07
4	197	Test Airport 197	Japan	Almaty Region	Istanbul	2026-09-29 19:07
5	196	Test Airport 196	Kazakhstan	London	Paris	2026-09-29 19:07
6	195	Test Airport 195	Kazakhstan	Tokyo	Tokyo	2026-09-29 19:07
7	194	Test Airport 194	Turkey	California	Paris	2026-09-29 19:07
8	193	Test Airport 193	France	California	Paris	2026-09-29 19:07
9	192	Test Airport 192	Japan	London	Almaty	2026-09-29 19:07
10	191	Test Airport 191	Japan	Almaty Region	Almaty	2026-09-29 19:07

Inserted random airport records are visible in the result set.

Result: 200 random airport rows were generated and inserted successfully.

Task 2

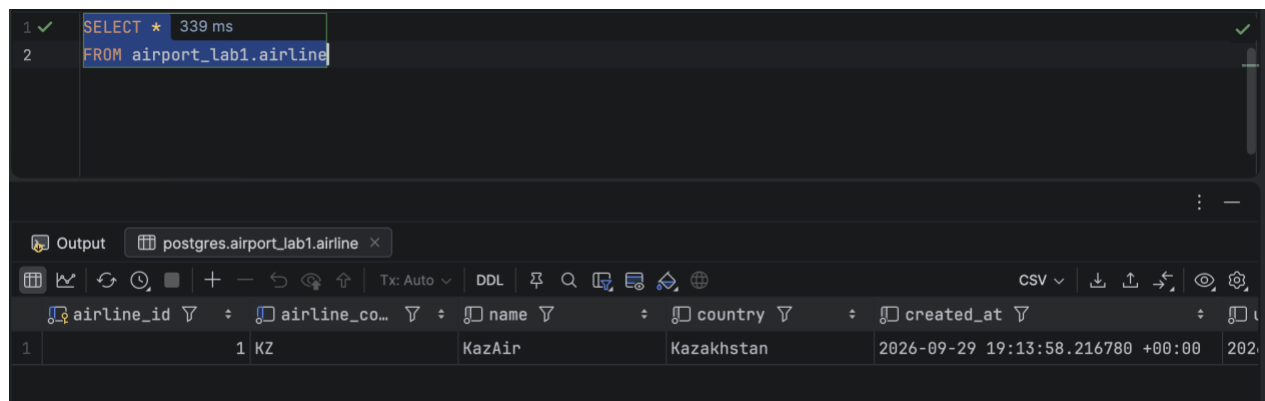
Add a new airline named "KazAir" based in "Kazakhstan" to the airline table.

SQL statement

```
INSERT INTO airport_lab1.airline ( 24 ms
    airline_code,
    name,
    country
)
VALUES (
    airline_code 'KZ',
    name 'KazAir',
    country 'Kazakhstan'
);
```

KazAir is inserted with airline code KZ and country Kazakhstan.

Result / after



The screenshot shows a database client interface. At the top, a SQL query is entered: `SELECT * FROM airport_lab1.airline`. Below the query, the results are displayed in a table. The table has five columns: `airline_id`, `airline_code`, `name`, `country`, and `created_at`. A single row is shown with the following values: `1`, `KZ`, `KazAir`, `Kazakhstan`, and `2026-09-29 19:13:58.216780 +00:00`.

airline_id	airline_code	name	country	created_at
1	KZ	KazAir	Kazakhstan	2026-09-29 19:13:58.216780 +00:00

The new airline record appears in the airline table.

Result: KazAir was added to the airline table.

Task 3

Update the airline country for "KazAir" to "Turkey".

SQL execution and result

```
1 UPDATE airport_lab1.airline
2 SET
3     country = 'Turkey',
4     updated_at = NOW()
5 WHERE name = 'KazAir'; 347 ms
```

Output postgres.airport_lab1.airline

	airline_id	airline_co...	name	country	created_at	
1	1	KZ	KazAir	Turkey	2026-09-29 19:13:58.216780 +00:00	202

The UPDATE targets KazAir and changes its country to Turkey.

Result: KazAir now has country = Turkey.

Task 4

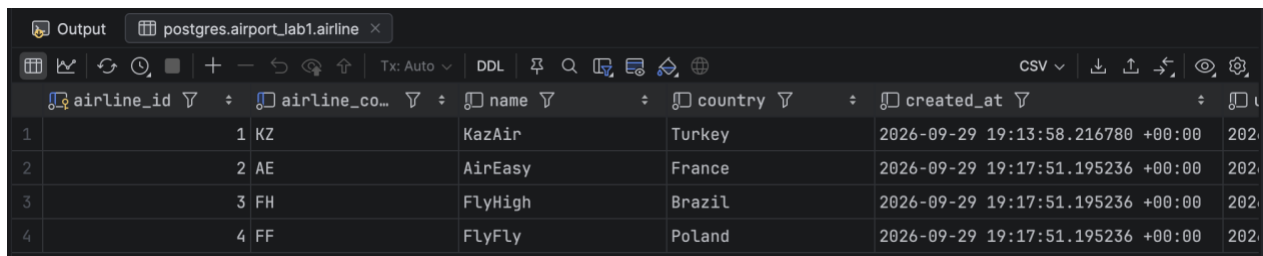
Add three airlines at once: AirEasy (France), FlyHigh (Brazil), and FlyFly (Poland).

SQL statement

```
1 ✓ INSERT INTO airport_lab1.airline ( 19 ms
2     airline_code,
3     name,
4     country
5 )
6 VALUES
7     ( airline_code 'AE', name 'AirEasy', country 'France' ),
8     ( airline_code 'FH', name 'FlyHigh', country 'Brazil' ),
9     ( airline_code 'FF', name 'FlyFly', country 'Poland' );
```

Three airline rows are inserted in a single INSERT statement.

Result / after



	airline_id	airline_code	name	country	created_at
1	1	KZ	KazAir	Turkey	2026-09-29 19:13:58.216780 +00:00
2	2	AE	AirEasy	France	2026-09-29 19:17:51.195236 +00:00
3	3	FH	FlyHigh	Brazil	2026-09-29 19:17:51.195236 +00:00
4	4	FF	FlyFly	Poland	2026-09-29 19:17:51.195236 +00:00

AirEasy, FlyHigh, and FlyFly are visible in the airline table.

Result: All three airlines were inserted successfully in one operation.

Task 5

Delete all flights whose scheduled arrival is in the year 2024.

SQL execution — part 1

```
1 ✓ BEGIN; 17 ms
2
3 ✓ CREATE TEMP TABLE tmp_flights_2024 29 ms
4     ON COMMIT DROP AS
5   SELECT flight_id
6     FROM airport_lab1.flight
7    WHERE scheduled_arrival_time >= '2024-01-01'
8          AND scheduled_arrival_time < '2025-01-01';
9
10 ✓ CREATE TEMP TABLE tmp_bookings_2024 20 ms
11     ON COMMIT DROP AS
12   SELECT booking_id
13     FROM airport_lab1.booking
14    WHERE flight_id IN (
15        SELECT flight_id FROM tmp_flights_2024
16      );
17
18 ✓ DELETE FROM airport_lab1.baggage_check 20 ms
19   WHERE booking_id IN (
20       SELECT booking_id FROM tmp_bookings_2024
21     );
22
23 ✓ DELETE FROM airport_lab1.boarding_pass 17 ms
24   WHERE booking_id IN (
25       SELECT booking_id FROM tmp_bookings_2024
26     );
27
28 ✓ DELETE FROM airport_lab1.booking_change 18 ms
29   WHERE booking_id IN (
30       SELECT booking_id FROM tmp_bookings_2024
31     );
32 |
33 ✓ DELETE FROM airport_lab1.baggage 16 ms
34   WHERE booking_id IN (
35       SELECT booking_id FROM tmp_bookings_2024
36     );
```

A transaction creates temporary ID lists and removes dependent records first.

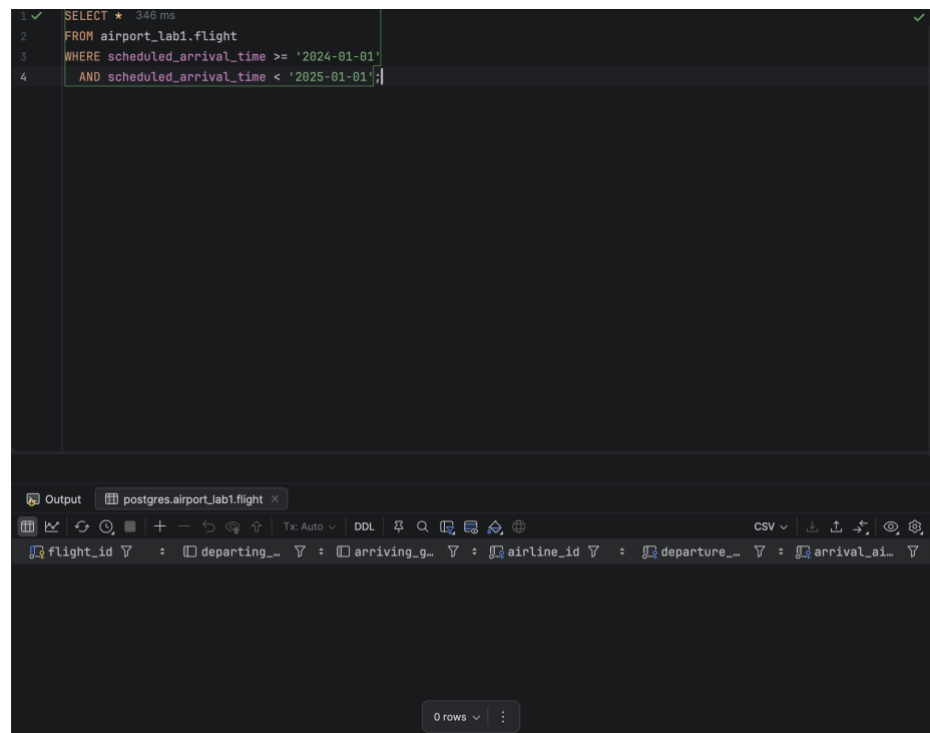
Task 5 — continued

SQL execution — part 2

```
37
38 ✓ DELETE FROM airport_lab1.booking 48 ms
39 WHERE booking_id IN (
40     SELECT booking_id FROM tmp_bookings_2024
41 );
42
43 ✓ DELETE FROM airport_lab1.flight 19 ms
44 WHERE flight_id IN (
45     SELECT flight_id FROM tmp_flights_2024
46 );
47
48 ✓ COMMIT; 19 ms
49
```

Bookings and then the matching 2024 flights are deleted; the transaction is committed.

Verification / after



The screenshot shows a SQL client interface with a query editor and a results pane. The query editor contains the following SQL statement:

```
1 ✓ SELECT * 346 ms
2 FROM airport_lab1.flight
3 WHERE scheduled_arrival_time >= '2024-01-01'
4 AND scheduled_arrival_time < '2025-01-01';
```

The results pane shows the output of the query. The table has columns: flight_id, departing_time, arriving_time, airline_id, departure_date, and arrival_date. The results pane is empty, indicating 0 rows returned.

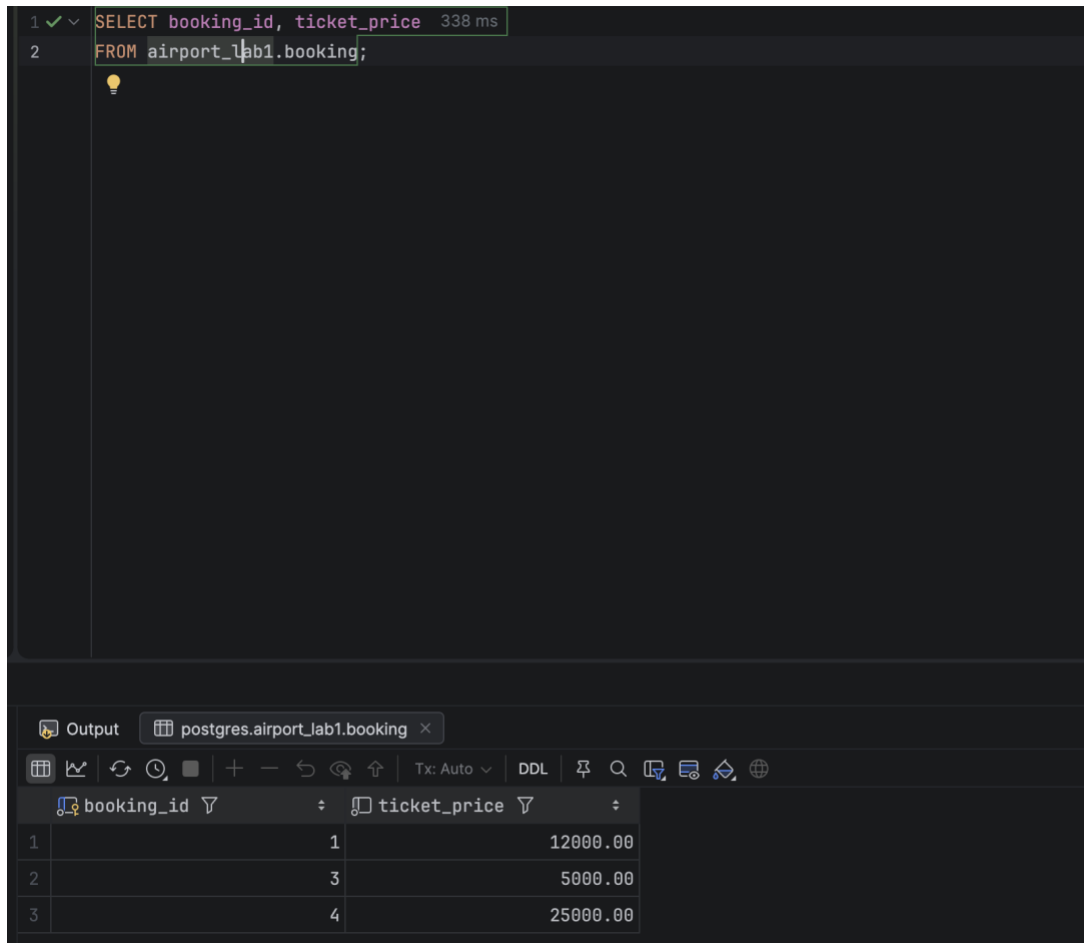
A query for scheduled arrivals in 2024 returns 0 rows.

Result: No flights with scheduled arrival dates in 2024 remain.

Task 6

Increase the price of all tickets in the booking table by 15%.

Before



The screenshot shows a SQL IDE with a query editor and an output window. The query editor contains the following SQL statement:

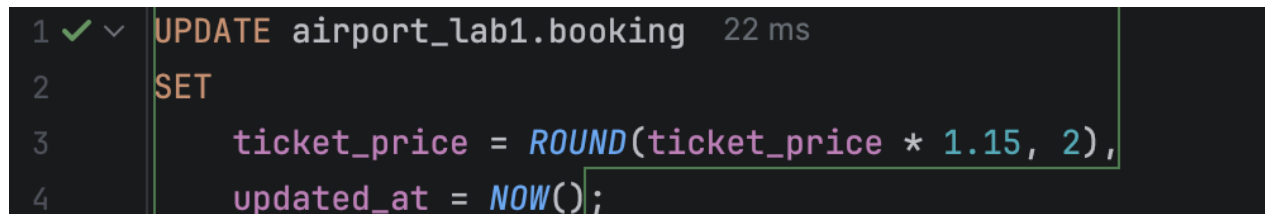
```
1 ✓ SELECT booking_id, ticket_price 338 ms
2 FROM airport_lab1.booking;
```

The output window displays the results of the query in a table format:

	booking_id	ticket_price
1	1	12000.00
2	3	5000.00
3	4	25000.00

Ticket prices before the 15% increase.

SQL statement



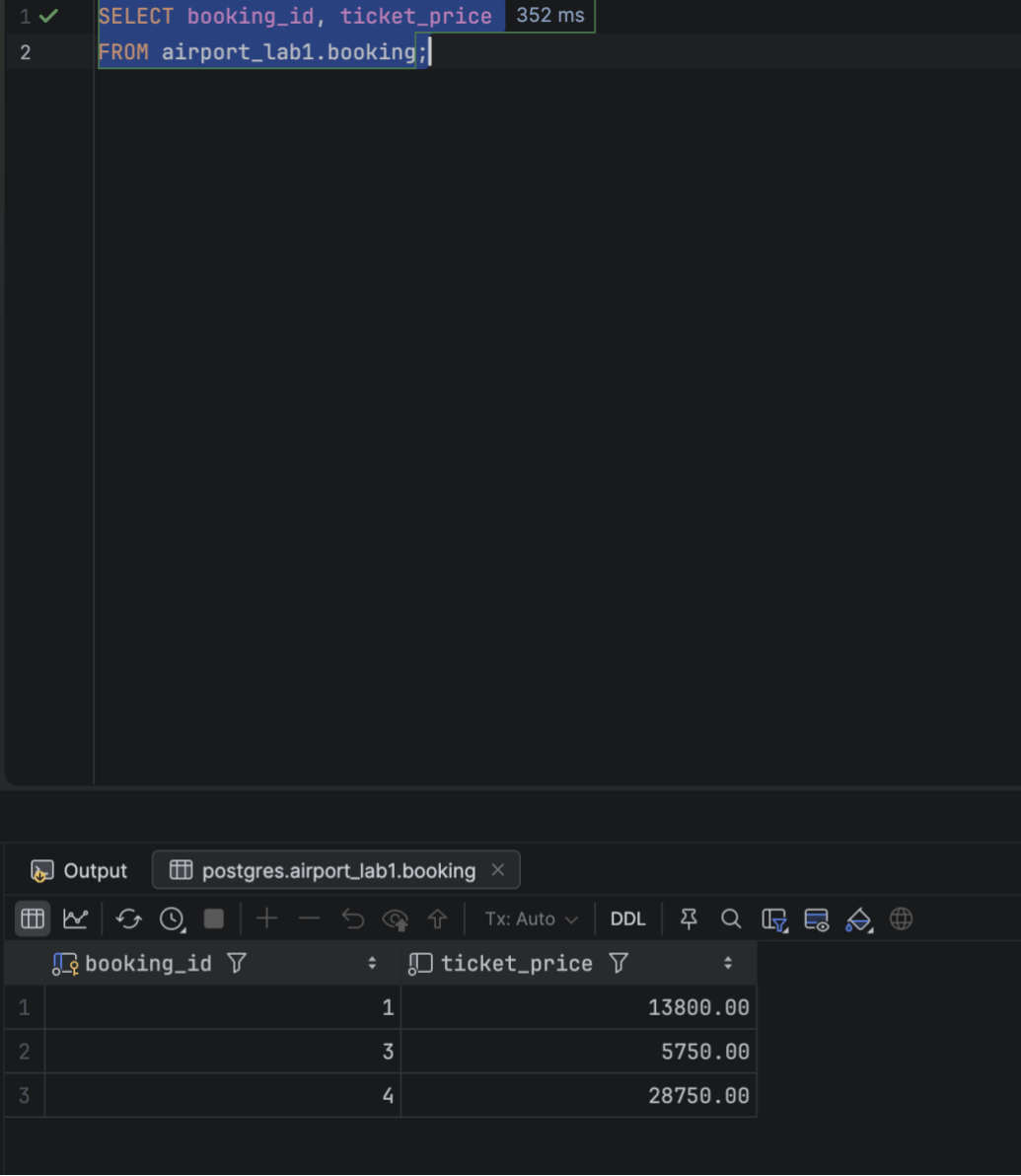
The screenshot shows a SQL IDE with the following SQL statement:

```
1 ✓ UPDATE airport_lab1.booking 22 ms
2 SET
3     ticket_price = ROUND(ticket_price * 1.15, 2),
4     updated_at = NOW();
```

ticket_price is multiplied by 1.15 and rounded to two decimal places.

Task 6 — after

After



The screenshot shows a PostgreSQL query editor with two lines of SQL code:

```
1 ✓ SELECT booking_id, ticket_price 352 ms
2 FROM airport_lab1.booking;
```

Below the query editor, the 'Output' tab is selected, showing the results of the query in a table format. The table has two columns: 'booking_id' and 'ticket_price'.

	booking_id	ticket_price
1	1	13800.00
2	3	5750.00
3	4	28750.00

Prices changed from 12000 → 13800, 5000 → 5750, and 25000 → 28750.

Result: All remaining booking ticket prices were increased by 15%.

Task 7

Delete all tickets where the price is less than 10000.

SQL execution

```
1 ✓ BEGIN; 11 ms
2
3 ✓ CREATE TEMP TABLE tmp_cheap_bookings 12 ms
4     ON COMMIT DROP AS
5     SELECT booking_id
6     FROM airport_lab1.booking
7     WHERE ticket_price < 10000;
8
9 ✓ DELETE FROM airport_lab1.baggage_check 10 ms
10 WHERE booking_id IN (
11     SELECT booking_id FROM tmp_cheap_bookings
12 );
13
14 ✓ DELETE FROM airport_lab1.boarding_pass 10 ms
15 WHERE booking_id IN (
16     SELECT booking_id FROM tmp_cheap_bookings
17 );
18
19 ✓ DELETE FROM airport_lab1.booking_change 12 ms
20 WHERE booking_id IN (
21     SELECT booking_id FROM tmp_cheap_bookings
22 );
23
24 ✓ DELETE FROM airport_lab1.baggage 8 ms
25 WHERE booking_id IN (
26     SELECT booking_id FROM tmp_cheap_bookings
27 );
28
29 ✓ DELETE FROM airport_lab1.booking 9 ms
30 WHERE booking_id IN (
31     SELECT booking_id FROM tmp_cheap_bookings
32 );
33
34 ✓ COMMIT; 10 ms
```

A temporary table identifies bookings below 10000; dependent rows are removed before deleting the bookings.

Result: Bookings with ticket_price < 10000 were deleted while respecting foreign-key dependencies.

Task 8

Set `airline_code` to "UNK" where `airline_code` is `NULL`.

SQL statement

```
1 ✓ UPDATE airport_lab1.airline 19 ms
2 SET
3     airline_code = 'UNK',
4     updated_at = NOW()
5 WHERE airline_code IS NULL;
```

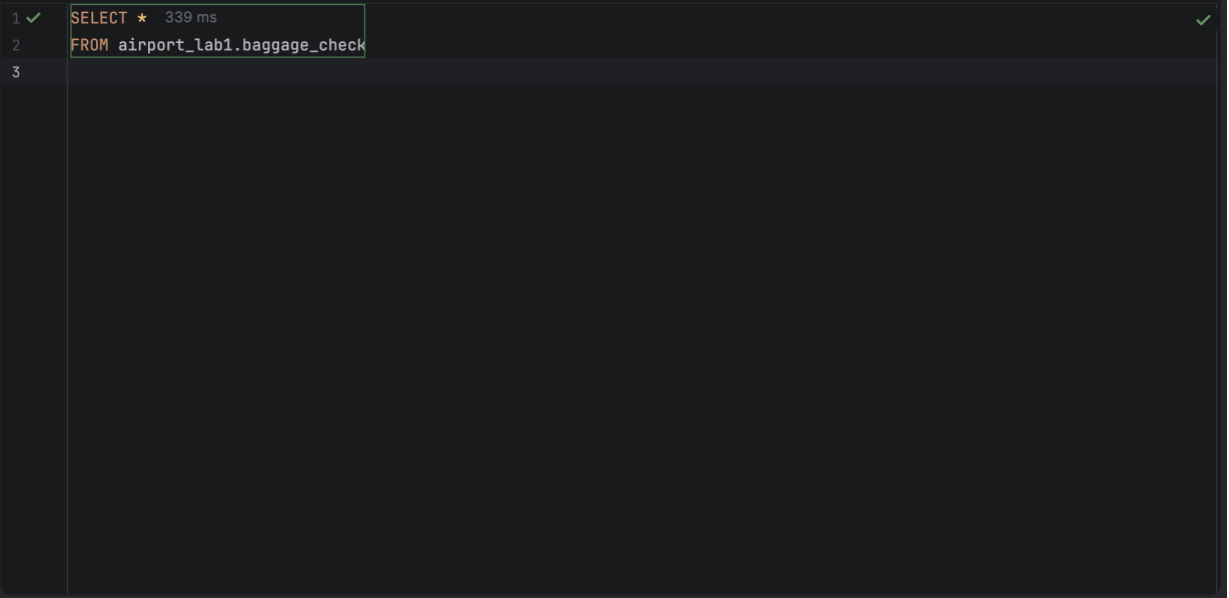
Rows with `airline_code` IS `NULL` are assigned the value `UNK`.

Result: The requested UPDATE statement was executed. In this schema `airline_code` is defined as NOT NULL, so NULL rows may not exist.

Task 9

Delete baggage check records created before June 1, 2023 whose result is "Not checked".

Before



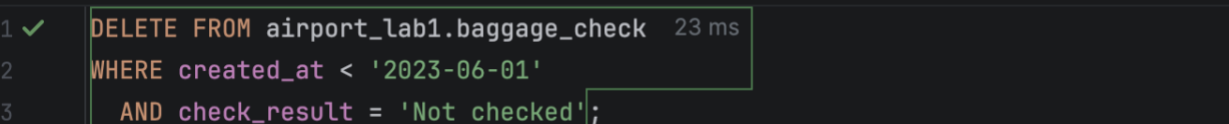
The screenshot shows a PostgreSQL query editor with a SELECT statement executed. The query is: `SELECT * FROM airport_lab1.baggage_check`. The results are displayed in a table with 6 columns: baggage_ch..., baggage_id, booking_id, passenger..., check_resu..., and created_at. There are 2 rows of data.

	baggage_ch...	baggage_id	booking_id	passenger...	check_resu...	created_at
1		1	1		Not checked	2023-05-01 05:00:00
2		2	2	4	Not checked	2024-03-15 07:00:00

At the bottom of the table, it says "2 rows".

Two baggage checks are shown as Not checked, including one from May 2023.

SQL statement



The screenshot shows a PostgreSQL query editor with a DELETE statement executed. The query is: `DELETE FROM airport_lab1.baggage_check WHERE created_at < '2023-06-01' AND check_result = 'Not checked';`

Delete Not checked records created before 2023-06-01.

Task 9 — after

After

The screenshot shows a PostgreSQL query editor with a successful query execution. The query is:

```
1 ✓ SELECT * 334 ms
2 FROM airport_lab1.baggage_check
3
```

The result is displayed in a table with the following columns: baggage_id, booking_id, passenger_name, check_result, and created_at. The table contains one row:

baggage_id	booking_id	passenger_name	check_result	created_at
2	4		Not checked	2024-03-15 07:00:00

The interface also shows a toolbar with various icons and a status bar indicating 1 row.

Only the later Not checked baggage-check record remains.

Result: The old unchecked baggage inspection was deleted successfully.

Task 10

Delete airport entries where state is NULL and city is either Mlawe or Kepuh.

SQL execution and result

```
1 ✓ DELETE FROM airport_lab1.airport 29 ms
2 WHERE state IS NULL
3 AND city IN ('Mlawe', 'Kepuh');
4
```

```
[2026-09-30 00:48:15:073] > SELECT *
      FROM airport_lab1.airport
[2026-09-30 00:48:15:419] 205 rows retrieved starting from 1 in 346 ms (execution: 3 ms, fetching: 343 ms)
[2026-09-30 00:48:54:608] > DELETE FROM airport_lab1.airport
      WHERE state IS NULL
      AND city IN ('Mlawe', 'Kepuh')
[2026-09-30 00:48:54:612] 2 rows affected in 4 ms
```

DELETE targets NULL state rows in Mlawe or Kepuh; the output reports 2 rows affected.

Result: Two matching airport records were deleted.

Task 11

Insert a new unchecked baggage inspection and return the generated baggage_check_id and created_at.

SQL execution and RETURNING result

```
1 ✓ INSERT INTO airport_lab1.baggage_check ( 352 ms
2     baggage_id,
3     booking_id,
4     passenger_id,
5     check_result,
6     created_at,
7     updated_at
8 )
9 SELECT
10     bg.baggage_id,
11     bg.booking_id,
12     bk.passenger_id,
13     check_result 'Not checked',
14     created_at NOW(),
15     updated_at NOW()
16 FROM airport_lab1.baggage AS bg
17     JOIN airport_lab1.booking AS bk
18     ON bg.booking_id = bk.booking_id
19 ORDER BY bg.baggage_id
20 LIMIT 1
21 RETURNING
22     baggage_check_id,
23     created_at;
```

Output Result 51

	baggage_check_id	created_at
1	3	2026-09-29 19:50:36.890221 +00:00

1 row

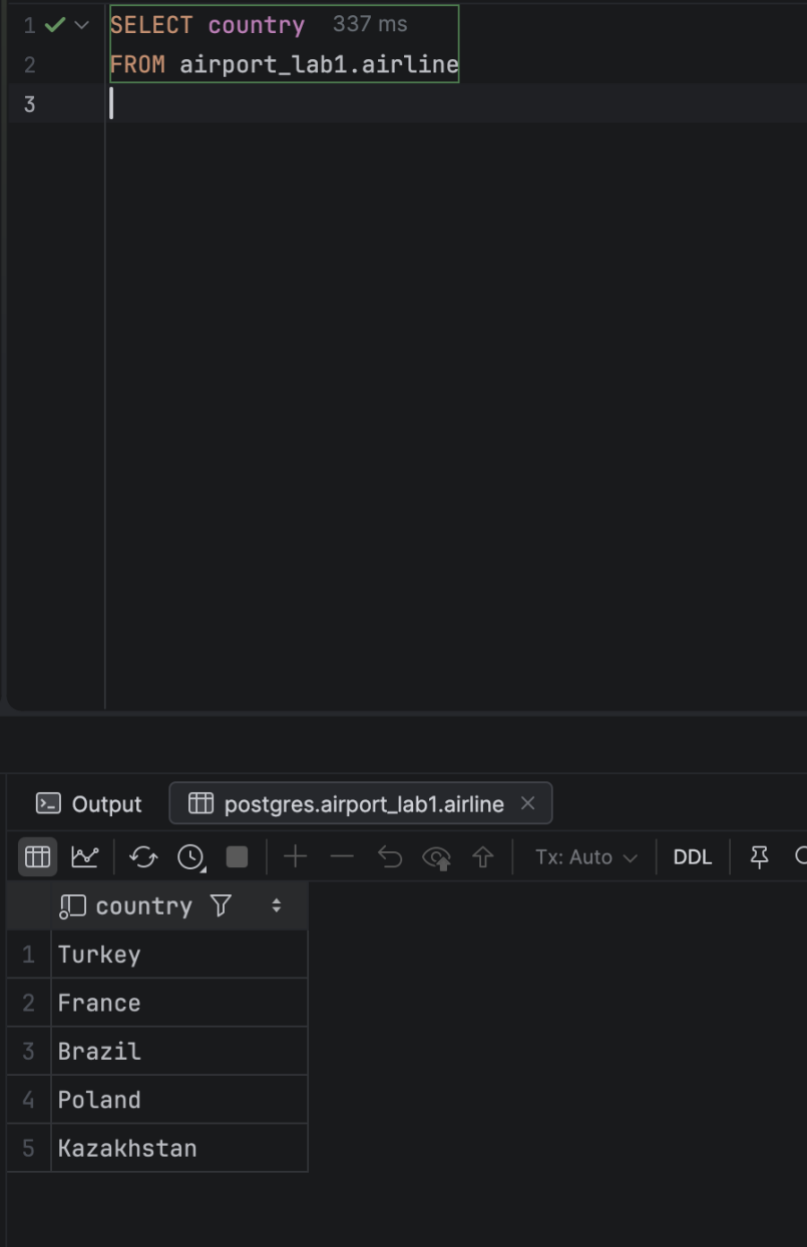
A new Not checked baggage inspection is inserted using existing baggage/booking data and RETURNING displays the generated values.

Result: The new baggage_check_id and created_at timestamp were returned immediately after insertion.

Task 12

Convert all country names in the airline table to uppercase.

Before



The screenshot shows a PostgreSQL query editor with a SQL query and its results. The query is:

```
1 ✓ SELECT country 337 ms
2 FROM airport_lab1.airline
3
```

The output is displayed in a table with the following data:

	country
1	Turkey
2	France
3	Brazil
4	Poland
5	Kazakhstan

Airline country names before case normalization.

Task 12 — after

After

```
1  UPDATE airport_lab1.airline
2  SET
3      country = UPPER(country),
4      updated_at = NOW();
```

Output postgres.airport_lab1.airline

country

1	TURKEY
2	FRANCE
3	BRAZIL
4	POLAND
5	KAZAKHSTAN

UPPER(country) converts all shown airline countries to uppercase.

Result: Airline country values were normalized to uppercase.

Task 13

Update `airline_id = 5` to Global Airways, United Kingdom, and set `updated_at` to the current time.

SQL execution and result

```
1 UPDATE airport_lab1.airline
2 SET
3     name = 'Global Airways',
4     country = 'United Kingdom',
5     updated_at = NOW()
6 WHERE airline_id = 5; 344 ms
```

Output postgres.airport_lab1.airline

country

1	TURKEY
2	FRANCE
3	BRAZIL
4	POLAND
5	United Kingdom

airline_id = 5 is changed to Global Airways, United Kingdom, with updated_at = NOW().

Result: The specified airline record was updated.

Task 14

Set state to "Capital District" for airports in Astana, London, and Tokyo.

SQL execution

```
1 ✓ UPDATE airport_lab1.airport 16 ms
2 SET
3     state = 'Capital District',
4     updated_at = NOW()
5 WHERE city IN (
6     'Astana',
7     'London',
8     'Tokyo'
9 );
```

Output postgres.airport_lab1.airport

	state
1	London
2	Tokyo
3	London
4	London
5	California
6	London
7	California
8	Almaty Region
9	Tokyo
10	Almaty Region
11	Almaty Region
12	London
13	London
14	California
15	London
16	Tokyo
17	London

203 rows

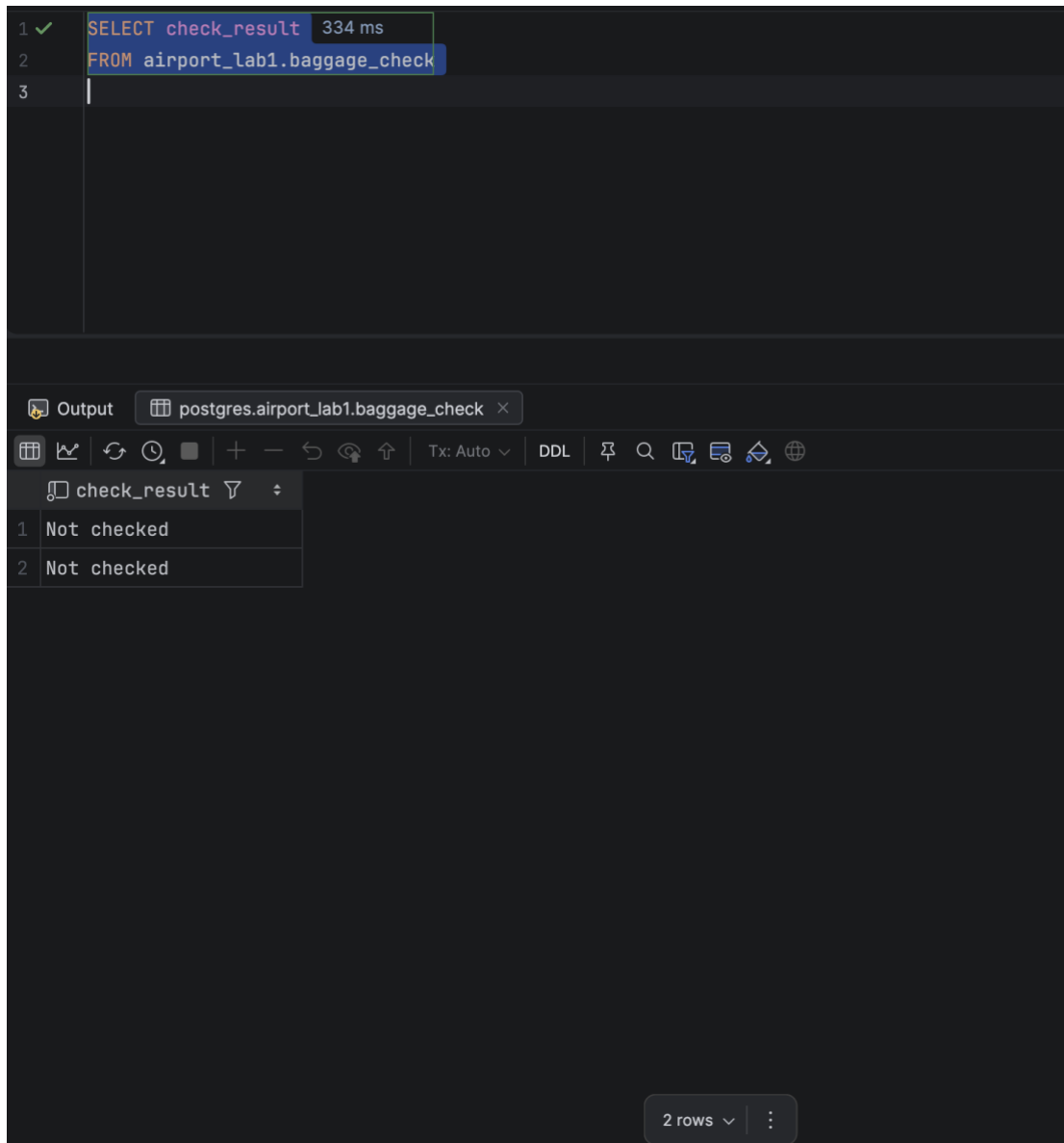
The UPDATE sets state = Capital District for airports whose city is Astana, London, or Tokyo.

Result: The statement targeted the required three cities and updated updated_at at the same time.

Task 15

Mark March 2024 baggage checks with status "Not checked" as "Checked".

Before



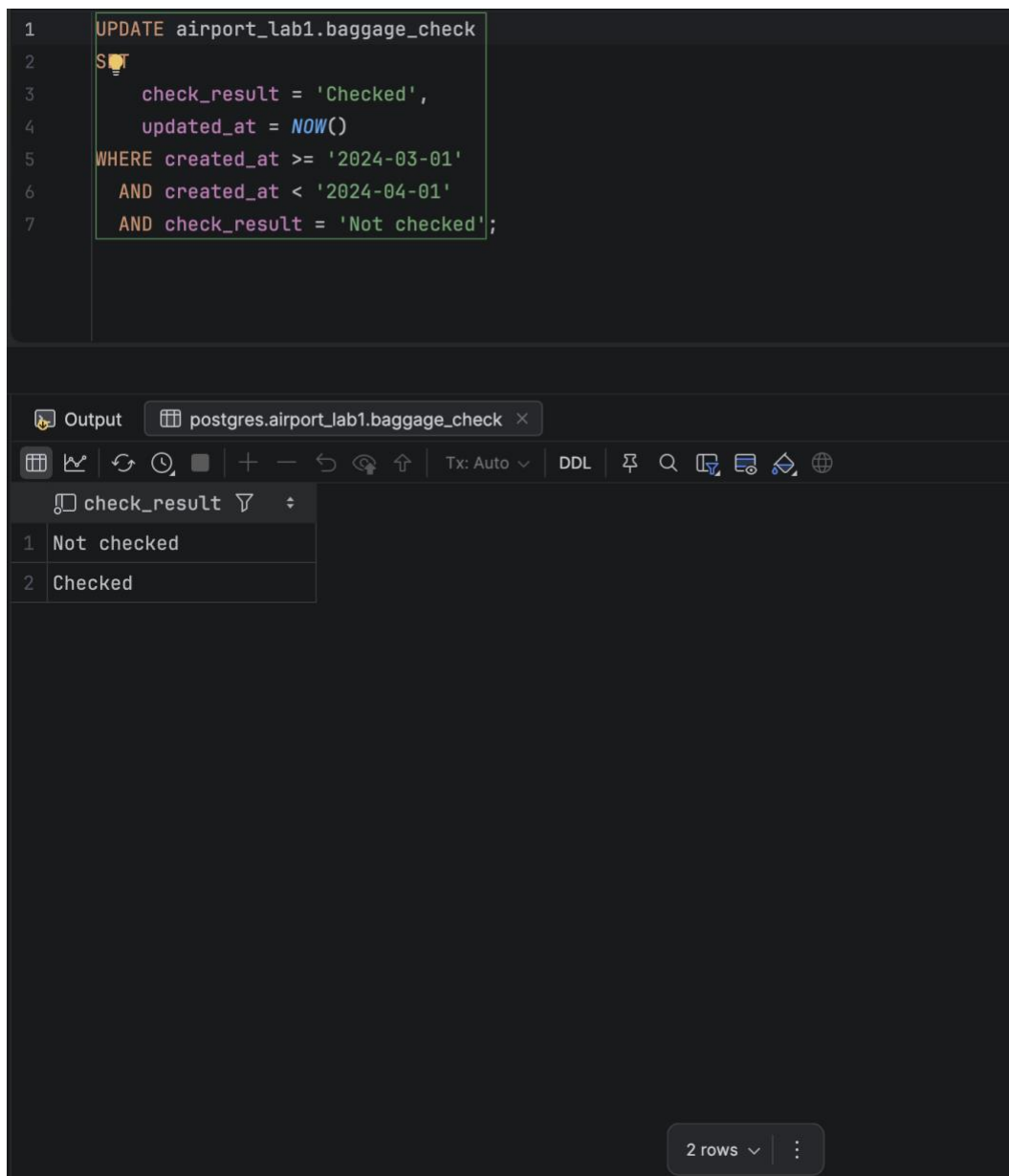
The screenshot shows a PostgreSQL client interface. The top pane displays a SQL query: `SELECT check_result FROM airport_lab1.baggage_check`. The query execution time is 334 ms. The bottom pane shows the results of the query, which are two rows, both with the value "Not checked" in the `check_result` column.

	check_result
1	Not checked
2	Not checked

Both baggage-check rows are shown as Not checked before the update.

Task 15 — after

After



The screenshot shows a PostgreSQL IDE interface. The top pane displays an SQL UPDATE statement:

```
1 UPDATE airport_lab1.baggage_check
2
3     check_result = 'Checked',
4     updated_at = NOW()
5 WHERE created_at >= '2024-03-01'
6     AND created_at < '2024-04-01'
7     AND check_result = 'Not checked';
```

The bottom pane shows the output of the query, titled "postgres.airport_lab1.baggage_check". It displays a table with two rows:

	check_result
1	Not checked
2	Checked

At the bottom right of the output pane, it indicates "2 rows" and a vertical ellipsis icon.

The March 2024 Not checked record is changed to Checked.

Result: The March 2024 unchecked baggage inspection was marked as Checked.

Conclusion

The screenshots document the execution of all 15 DML tasks in the airport_lab1 schema. Before/after evidence is shown where it was captured, and each section is arranged in assignment order for easier review and defense.